AI AUDIT WORKBENCH

Architecture Plan — Next.js Frontend

Architecture objective
Build a demo-ready agentic audit workbench with a polished Next.js interface, a Python-based audit backend, human approval, traceable evidence, and a deployable cloud prototype.

1. High-Level Architecture

LAYER	COMPONENTS	RESPONSIBILITY
Presentation	Next.js + React + TypeScript Tailwind CSS + shadcn/ui React Flow (@xyflow/react)	Chat interface, workflow canvas, approval UI, execution status, findings and evidence panels.
API	FastAPI REST endpoints SSE for run progress	Validate requests, enforce approval, start and query runs, stream status updates.
Orchestration	LangGraph workflow Planner + execution + explanation steps	Maintain workflow state, control step order, pause for approval, resume execution.
Audit logic	Python + Pandas Deterministic rules and validations	Validate input data, run checks, calculate outcomes, record exceptions.
AI / retrieval	Google Gemini API ChromaDB policy index (optional MVP)	Generate structured plans and grounded explanations; retrieve relevant policy text where needed.
Persistence	MongoDB Evidence files in controlled storage	Store runs, plan versions, approvals, findings, evidence metadata and execution traces.
Deployment	Google Cloud Run GitHub + secrets configuration	Host the frontend and backend; keep credentials outside source control.

Architecture flow

User → Next.js UI → FastAPI → LangGraph → Gemini / Python audit checks / policy retrieval → MongoDB and evidence builder → API updates → Next.js findings and report.

Key design principle

Use Gemini for planning and explanation, but use deterministic Python rules for audit decisions. Enforce human approval in the backend, not only in the frontend.

2. Frontend Architecture — Next.js

Recommended frontend stack

TECHNOLOGY	USE
Next.js + TypeScript	Application shell, routing, reusable components and typed API integration.
Tailwind CSS + shadcn/ui	Consistent styling, dialogs, tabs, forms, tables and accessible UI primitives.
React Flow	Visual workflow graph with nodes, edges, dependencies and execution states.
Fetch / TanStack Query	Call the FastAPI backend and manage loading, errors, caching and run status.
SSE	Receive server-to-browser progress updates for long-running audit workflows.

Suggested page layout

- **Audit workspace:** left chat / objective panel; central workflow canvas; right-side run details or artifacts panel.
- **Plan review:** show the proposed steps, inputs, expected outputs and approval / edit actions.
- **Execution:** visually distinguish pending, running, completed and failed nodes; show concise logs and timestamps.
- **Findings:** list severity, rule, source record, result, explanation and evidence link.
- **Report:** provide a downloadable report and a concise run summary.

Frontend-to-backend contract

Keep the frontend focused on presentation. The browser sends objectives, approvals and queries to FastAPI; the backend owns plan validation, execution permissions, workflow state and evidence persistence.

3. Backend Modules and API Endpoints

METHOD	ENDPOINT	PURPOSE
POST	/api/audits/plan	Create a run and generate a structured plan.
POST	/api/audits/{run_id}/approve	Record an approval against the current plan version.
POST	/api/audits/{run_id}/execute	Start execution only when the plan is approved.
GET	/api/audits/{run_id}	Return status, plan summary and findings.
GET	/api/audits/{run_id}/events	SSE stream for node status and progress events.
GET	/api/audits/{run_id}/evidence	Return evidence metadata and source references.
GET	/api/audits/{run_id}/report	Generate or download the report.
GET	/health	Health check for deployment monitoring.

4. Workflow and Data Lifecycle

1. Objective and input

User submits a review objective and a sample CSV/JSON/document. Validate file type, size and schema.

2. Plan generation

Gemini returns structured JSON. Validate it against a Pydantic schema and allow only approved node types and tools.

3. Approval

Persist the plan version and require explicit approval. The backend rejects execution requests without approval.

4. Execution

LangGraph runs the workflow. Python applies deterministic checks; optional retrieval provides relevant policy clauses.

5. Findings and explanations

Store raw check outcomes first. Ask Gemini to explain those outcomes using supplied evidence; do not let it invent results.

6. Evidence and report

Record source references, rule/version, timestamps, output hashes where useful, and the final report location.

5. MongoDB Collections

COLLECTION	KEY FIELDS
audit_runs	run_id, objective, status, plan_version, approval, timestamps, input reference, configuration
audit_findings	finding_id, run_id, severity, rule_id, source_record_ref, observed_value, expected_value, explanation
audit_evidence	evidence_id, run_id, source_ref, policy_ref, check_result, timestamp, content_hash
audit_events	run_id, event_type, node_id, status, timestamp, safe metadata

6. Deployment Plan

- **Frontend:** deploy the Next.js app to Cloud Run. Configure the API base URL through environment variables.
- **Backend:** deploy FastAPI and its Python dependencies to Cloud Run. Keep Gemini credentials in Secret Manager or secure environment configuration.
- **Database:** use a reachable MongoDB deployment with network access restricted appropriately. Avoid relying on container-local files for persistent data.
- **Policy index:** for the MVP, use a small prebuilt ChromaDB index only if persistence and deployment are manageable; otherwise start with a small policy file and deterministic lookup.
- **CI / repository:** publish the source to GitHub with a README, architecture diagram, setup instructions and .env.example containing placeholders only.

Deployment topology

Browser → Next.js on Cloud Run → FastAPI on Cloud Run → Gemini API / audit engine / MongoDB. The backend emits SSE events to the browser through the API service.

7. Four-Day Implementation Plan

DAY	PRIORITY	DONE WHEN
1	Backend skeleton, Gemini plan output, deterministic checks, sample dataset, basic Next.js shell.	A request generates a validated plan and expected findings.
2	React Flow canvas, plan approval, findings/evidence UI, report output, API integration.	A teammate can complete the workflow through the UI.
3	Cloud Run deployment, MongoDB connectivity, SSE progress, GitHub README and smoke tests.	Public app and repository are accessible and tested.

DAY	PRIORITY	DONE WHEN
4	Bug fixes, feature freeze, demo rehearsal and recording, pitch deck and final checklist.	All submission deliverables are ready.

8. Suggested Repository Structure

```
ai-audit-workbench/
■■■ frontend/
■  ■■■ app/
■  ■■■ components/
■  ■  ■■■ chat/
■  ■  ■■■ workflow-canvas/
■  ■  ■■■ findings/
■  ■  ■■■ evidence/
■  ■■■ lib/api-client.ts
■  ■■■ package.json
■■■ backend/
■  ■■■ app/
■  ■  ■■■ main.py
■  ■  ■■■ api/
■  ■  ■■■ agents/
■  ■  ■■■ workflow/
■  ■  ■■■ audit_engine/
■  ■  ■■■ retrieval/
■  ■  ■■■ evidence/
■  ■  ■■■ reports/
■  ■  ■■■ db/
■  ■■■ requirements.txt
■  ■■■ Dockerfile
■■■ sample_data/
■■■ docs/
■  ■■■ architecture.md
■■■ .env.example
■■■ README.md
```

9. Security and Reliability Guardrails

- Never expose Gemini or database credentials to the browser; all model calls happen in FastAPI.
- Validate uploaded files, allowed workflow node types, model output schemas and all API inputs.
- Enforce approval and access checks on the backend; never trust frontend state alone.
- Keep audit event logs useful but avoid storing secrets or unnecessary sensitive source data.
- Use least-privilege credentials, HTTPS, restricted database access and bounded upload sizes.
- Show clear run states and recoverable errors; test the complete workflow using a clean browser session.

10. MVP vs Later Enhancements

MVP — 4 days	Later — after the hackathon
One workflow, one sample dataset, Gemini planner, deterministic checks, human approval, evidence-backed findings, basic report, Cloud Run deployment.	More agent specializations, multi-jurisdiction rule packs, enterprise identity and roles, persistent vector infrastructure, richer observability, integrations and scalable worker queues.

Success criteria: A user can enter an objective in the Next.js UI, review and approve a generated plan, run deterministic checks, inspect evidence-backed findings, and download a report from the deployed prototype.